

AprilCube: 3D-Printable Fiducial Cubes for Reliable 6-DoF Pose Estimation

Younghyo Park

AprilCube Project

<https://github.com/younghyopark/aprilcube>

Abstract—AprilCube is an open-source toolkit for generating 3D-printable cuboids with fiducial markers on all six faces and estimating their six degree-of-freedom pose from camera images. The repository couples a parametric generator for multi-material 3MF printing with a detector that uses the same configuration file to recover the three-dimensional corner position of every marker. This paper describes the software artifact, its geometry model, generated print and simulation assets, pose-estimation pipeline, streaming API, and synthetic validation harness. The project is intended as a reproducible physical tracking target for robotics, calibration, simulation, and interactive vision experiments.

Index Terms—fiducial marker, pose estimation, 3D printing, robotics, software

I. INTRODUCTION

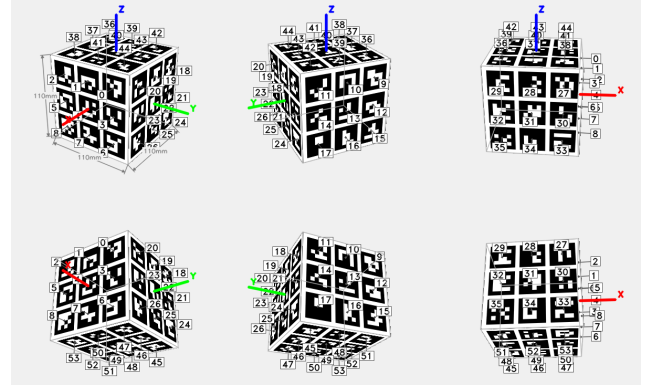
Planar fiducial systems such as ArUco and AprilTag provide robust local features for camera pose estimation [1], [2]. They are widely used because the printed target is cheap, the visual code carries an identifier, and off-the-shelf detectors can localize marker corners accurately. A single planar marker or board, however, exposes only one plane of three-dimensional structure. It can leave the camera view under rotation, become ambiguous when viewed from near-frontal single-marker configurations, and lose many correspondences when the target is partially occluded.

AprilCube addresses this by turning the marker board into a printable three-dimensional object. Markers are placed on every face of a known cuboid, and every marker corner is represented in a common object coordinate frame. As the object rotates, different faces become visible, but the detector can aggregate all visible corners into a single Perspective-n-Point (PnP) problem. The target therefore behaves less like a flat calibration sheet and more like a small tracked object that can be grasped, moved, placed in a robot workspace, or loaded into a simulator.

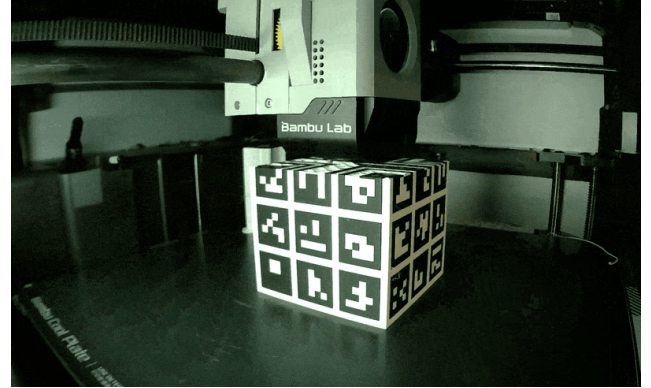
The repository contains two tightly coupled components. The generator produces printable cuboids, model previews, and simulation assets. The detector consumes the generated configuration and estimates the cuboid pose from images or video. This shared data model is the central design choice: the printed artifact, the detector, and the simulation mesh are all derived from the same marker dictionary, grid, physical cell size, margins, borders, and tag identifiers.

The main contributions are:

- a parametric generator for ArUco/AprilTag cuboids with configurable grids, tag sizes, margins, borders, and marker identifiers;



(a) Generated model preview



(b) 3D printing still

Fig. 1. AprilCube couples a generated fiducial cuboid model with a printable physical artifact. (a) The generator renders marker identifiers and object-frame axes. (b) A still frame from the repository GIF shows the marker pattern being fabricated on a dual-material FDM printer.

- an asset pipeline that emits multi-color 3MF files, detector JSON, preview images, textured OBJ assets, and MuJoCo MJCF models from the same geometry;
- a Python detector that reconstructs exact 3D marker-corner positions, aggregates multi-face observations, solves robust PnP, filters video poses, and exposes synchronous, asynchronous, and visualization-oriented APIs;
- a synthetic benchmark harness that renders ground-truth cuboid poses, applies degradations such as occlusion and blur, and checks pose error across systematic viewpoints and trajectories.

II. DESIGN GOALS

AprilCube was designed around the practical workflow of making a physical target, printing it, and then using it in camera-driven robotics software without manually transcribing geometry. The first goal is self-description: every printed object has a configuration file that fully specifies the dictionary, tag identifiers, dimensions, and face layouts. The second goal is consistency across media: the 3MF, preview image, OBJ mesh, MJCF model, and detector all use the same object coordinate frame. The third goal is graceful degradation in video: the detector should provide useful estimates when only a subset of faces is visible, when one or two detections are poor, or when a short burst of frames is temporarily lost.

These goals make the repository closer to a reproducible instrument than to a collection of scripts. The user can regenerate a target, print an already-generated model, load the same target into simulation, and run the detector with the same JSON file. This is especially useful for small projects where tracking hardware, motion-capture systems, or carefully manufactured calibration rigs are too expensive or cumbersome, but where a planar printed marker is too fragile a pose reference.

The design also favors inspectability. Generated thumbnails display the marker identifiers, face geometry, and object axes before fabrication. The detector reports visible faces, tag ids, inlier counts, reprojection error, and a debug overlay in addition to the pose. Those diagnostics are intentionally simple because many failures in fiducial tracking are practical failures: the camera is out of focus, the print contrast is poor, the wrong configuration file was loaded, or the marker is viewed at an angle where only a small number of corners remain reliable.

III. REPOSITORY ARTIFACT

AprilCube is packaged as a Python project with a command-line entry point, a public Python API, generated model assets, examples, and synthetic tests. The core runtime dependencies are NumPy and OpenCV’s contributed ArUco module [3]. Optional visualization uses viser and the textured OBJ model exported by the generator.

The current repository includes 28 generated model directories. These cover cuboids from one to ninety total tags, with tag sizes from 12 mm to 30 mm and side lengths from 30 mm to 180 mm. Each model directory stores a multi-color 3MF file, a detector configuration file, a rendered preview, and MuJoCo-compatible assets. This model zoo is useful in two ways: users can print a target without first learning the generator options, and the checked-in outputs serve as examples of the exact file bundle produced by the code.

The public package interface is intentionally small. Users can call `aprilcube.detector()` with either a model directory or a configuration file, plus camera intrinsics as a JSON file, dictionary, or 3-by-3 matrix. The returned estimator processes OpenCV frames and reports pose, reprojection error, visible faces, detected tags, and optional visualization overlays. The command-line interface exposes generation and webcam-oriented workflows for users who do not need to write Python code.

TABLE I
MAIN REPOSITORY ARTIFACTS

Artifact	What it contributes
<code>generate.py</code>	Parametric cuboid geometry, marker textures, 3MF writing, thumbnail rendering, OBJ and MJCF export.
<code>detect.py</code>	Marker detection, 2D–3D aggregation, PnP pose solve, temporal filtering, overlays, async processing, and viser integration.
<code>models/*_cube/</code>	Pre-generated printable and simulation-ready cubes and cuboids.
<code>test_detection.py</code>	Synthetic renderer, discrete viewpoint checks, trajectories, video output, and image degradations.
<code>examples/</code>	Webcam and web-based visualization entry points.

IV. PARAMETRIC CUBE GENERATION

The generator takes a marker dictionary, tag identifiers, a grid size $g_x \times g_y \times g_z$, and either tag size or cell size. The grid describes how many markers lie along the object axes rather than how many markers lie on one particular face. This makes rectangular cuboids natural: a $5 \times 4 \times 1$ target, for example, becomes a flat box with large top and bottom faces and narrow side strips, while a $3 \times 3 \times 3$ target becomes a conventional cube.

A marker with dictionary bit width d occupies

$$m = d + 2 \quad (1)$$

cells after including its one-cell black border. With border b , inter-tag margin s , and n tags along one object axis, the total number of cells along that axis is

$$a(n) = 2b + nm + \max(0, n - 1)s. \quad (2)$$

The physical dimensions are then $c[a(g_x), a(g_y), a(g_z)]$, where c is the cell size in millimeters. The total number of required markers is

$$N = 2(g_x g_y + g_x g_z + g_y g_z). \quad (3)$$

The six faces are assigned stable outward normals and face-local “right” and “down” directions. The convention is chosen so that the cross product of the face-local right and down axes equals the outward normal. This convention is used in three places: texture layout, mesh triangle winding, and detector-side corner reconstruction. Because all three are derived from the same face definitions, the rendered preview, the exported simulation mesh, and the detector agree on the object frame.

The generator writes a configuration JSON that maps every marker identifier to a face and stores the physical geometry needed by the detector. This file is the contract between fabrication and perception. It includes the dictionary name, grid, tag identifiers, per-face id layout, cell size, tag size, margins, borders, and box dimensions. The detector does not infer the printed geometry from the image; it reconstructs it exactly from this JSON.

For physical fabrication, the project writes 3MF files with Bambu Studio paint-color attributes so white cells can be

assigned to a second extruder on dual-material printers. The mesh is constructed at cell resolution: black cells are part of the base material, while white cells are marked with the alternate paint color. This keeps the output compatible with slicer workflows that expect a single 3MF package rather than separate STL shells.

For simulation and visualization, the same face textures are packed into a 3-by-2 texture atlas and exported as Wavefront OBJ, MTL, and MJCF files for MuJoCo [5]. The exported OBJ uses meter units and an object frame consistent with detector output. The MJCF file adds both a textured visual mesh and a transparent box collision geom, making the target convenient for simulated robotics scenes where the visual marker pattern and physical collision shape should be aligned.

V. FABRICATION WORKFLOW

The intended fabrication workflow begins with a grid and a dictionary choice. Small cubes such as $1 \times 1 \times 1$ or $2 \times 2 \times 2$ are compact physical props, while larger grids provide more visible tags and therefore more PnP correspondences. Flat boxes such as $5 \times 4 \times 1$ behave like thick marker boards with side markers that keep the object observable under roll or pitch. The generator checks that the requested dictionary contains enough unique identifiers and then writes the complete output directory.

The 3MF output is meant to be opened directly in a slicer that understands material assignment attributes. In a typical dual-color FDM workflow, the base material is black and the painted cells are white. The choice is conventional rather than mathematical: the detector ultimately sees image contrast. In practice, matte materials, adequate lighting, and a camera exposure that avoids clipping the white cells matter more than the exact brand of filament.

Because the marker pattern is part of the physical geometry, the generated dimensions should be treated as the design specification rather than as a guarantee of the printed object. Slicer settings, extrusion width, nozzle diameter, layer height, elephant-foot compensation, and surface finish can all perturb marker corners. The repository therefore emphasizes reproducibility of the digital artifact and leaves metrological validation to the user’s printer and camera setup.

VI. POSE ESTIMATION PIPELINE

The detector first loads the JSON configuration and reconstructs a map from marker id to its four three-dimensional corner coordinates. The corner order matches OpenCV’s marker convention, so image corners can be paired directly with object-space corners. For every frame, the detector converts the image to grayscale, applies contrast normalization, detects ArUco or AprilTag-compatible markers with tuned OpenCV parameters, filters detections to the cube’s known id set, and aggregates all observed marker corners into a single 2D–3D correspondence set.

The OpenCV detector parameters are tuned for the visual properties of multi-material 3D prints. The accurate preset uses sub-pixel corner refinement, a wide adaptive-threshold

window range, relaxed candidate perimeter limits, and dense perspective sampling. A faster preset reduces threshold passes and skips sub-pixel refinement for webcam use. The detector also includes a fallback pass with sharpening and relaxed decoding when the primary detector finds no valid markers, which helps recover frames affected by motion blur or poor contrast.

Given image points u_i , object points X_i , and camera matrix K , the pose is estimated by minimizing

$$\sum_i \|\pi(K(RX_i + t)) - u_i\|_2^2, \quad (4)$$

where R, t are the object-to-camera rotation and translation and π is perspective projection. With at least six points the implementation uses `solvePnP`; otherwise it uses direct `solvePnP`. The cold-start solver uses SQPnP [4], then the inlier pose is refined with Levenberg–Marquardt. Because visible markers can span multiple non-coplanar faces, the system often has a richer point distribution than a single planar target.

Several implementation details are included for real-time robustness. Detected quads are scored for area, convexity, and side-length balance before entering PnP. If a prior pose is available, rejected ArUco candidates can be compared against projected marker locations and recovered when their corners are geometrically plausible. After PnP, detections with unusually high per-tag reprojection error can be dropped and the pose re-solved. A face-normal check rejects flipped PnP solutions by verifying that visible face normals point toward the camera, and a temporal jump gate rejects sudden pose discontinuities that are inconsistent with the current velocity estimate.

VII. TEMPORAL TRACKING AND API

For streaming applications, AprilCube includes an error-state Kalman filter. Translation is modeled as position and velocity in millimeters. Rotation uses a nominal unit quaternion with a small-angle error state and angular velocity. This multiplicative formulation avoids filtering Euler angles and avoids treating quaternion components as independent Euclidean coordinates. After each correction, the small-angle rotation error is folded into the nominal quaternion and reset.

The process noise adapts to the current estimated speed: slow motion receives lower acceleration and angular-acceleration noise, while fast motion receives higher noise. Measurement covariance is scaled by reprojection error, tag count, and inlier ratio; a pose with many inlier markers and low reprojection error is trusted more than a pose from a small or noisy correspondence set. Mahalanobis gating down-weights soft outliers and reinitializes the filter after hard outliers or repeated inconsistencies. The filter prediction is also reused as a PnP initialization and as a short dropout fallback.

When direct marker detection fails, the detector can track previously observed corners with Lucas–Kanade optical flow. If the Kalman filter has a prediction, projected corner locations seed the optical-flow search. This path is intentionally

weighted less strongly than fresh marker detections during filtering.

The class also supports a swap-not-queue async mode for webcam loops. Submitted frames replace any pending frame, so the worker always processes the freshest available image instead of accumulating stale frames. For robotics and visualization, the estimator can convert camera-frame poses into a world frame through an optional extrinsic matrix. The viser integration displays the textured cube mesh, object axes, camera feed, and status metadata in a web scene while the user’s capture loop continues to call `process_frame()`.

VIII. SOFTWARE INTERFACE

The package-level interface is meant to hide the repository layout while preserving access to the underlying estimator. A typical program calls `aprilcube.detector(model, intrinsics)`, where `model` can be either a model directory or a `config.json` path. Intrinsic can be supplied as a calibration JSON file, a dictionary containing focal lengths and principal point, or a NumPy camera matrix. This keeps quick tests concise while still allowing calibrated camera parameters and distortion coefficients in real deployments.

Each call to `process_frame()` returns a dictionary rather than a custom opaque object. The dictionary contains a success flag, Rodrigues rotation vector, translation vector in millimeters, homogeneous camera-frame transform, detected tag ids, visible faces, inlier count, reprojection error, and a rendered debug visualization. This representation is deliberately easy to log, print, serialize, and inspect during experiments. When an extrinsic world transform is provided, `world_pose()` converts the detected object pose into a meter-scale world-frame transform.

The examples in the repository exercise three integration styles. A minimal webcam script shows direct OpenCV capture and frame-by-frame detection. A viser example demonstrates web-based 3D visualization with the generated textured mesh. The asynchronous API supports interactive applications where rendering, robot control, or user-interface work should continue even when marker detection takes longer than one camera period.

IX. SYNTHETIC VALIDATION

The test harness renders generated cubes at known six degree-of-freedom poses and then runs the same detector used for real frames. The renderer projects textured cube faces into an image using the configured camera matrix, performs back-face culling, and compares the recovered pose against ground truth. This provides a fast regression test for coordinate conventions, tag ordering, PnP behavior, and outlier handling.

The discrete benchmark covers 48 viewpoints: face-on views, edge and corner views, and oblique elevations. The renderer can also create orbit, spiral, approach, tumble, translation, shake, wander, and stress videos with optional occlusion and motion blur. These trajectories are useful for visual inspection because the output video can include marker outlines, cube wireframes, pose error, and detection status.

TABLE II
REPRESENTATIVE SYNTHETIC CHECKS

Model	Condition	Det./pass	Mean rot./trans.
2x2x2, 30 mm	clean	48/48, 48/48	0.097 deg, 0.25 mm
3x3x3, 30 mm	clean	48/48, 48/48	0.086 deg, 0.28 mm
2x2x2, 30 mm	20% occ., 7 px blur	40/48, 38/48	4.845 deg, 0.47 mm

Table II reports representative checks run from the repository at 800 by 800 resolution without temporal filtering. These tests verify geometric consistency and detector behavior, but they are not a substitute for physical accuracy evaluation with calibrated cameras, real prints, lighting variation, and motion.

The multi-camera test file extends the synthetic setup to two cameras with a known relative transform. It renders primary and auxiliary views, moves the cube between the cameras, applies per-camera degradations, and compares primary-only tracking with a development multi-camera path. This code is useful as evidence of the intended direction of the project, but it is kept out of the main claims because the public package interface currently exposes a single-camera estimator.

X. LIMITATIONS AND INTENDED USE

AprilCube is a software and fabrication artifact, not a complete measurement system. Accuracy depends on camera calibration, print quality, marker contrast, focus, exposure, and the mechanical flatness of the printed faces. In particular, FDM artifacts such as layer lines, color bleeding, rounded cell corners, glossy surfaces, and slicer-specific material assignment can affect corner localization. The detector includes parameter choices intended for these conditions, but real-world accuracy should be validated for each printer, filament, camera, and lighting setup.

The included generated models target dual-color FDM workflows, although the geometry and detector are not specific to one printer. A different fabrication process could use the same JSON geometry as long as the printed marker pattern preserves scale and corner locations. Conversely, users who need very high metrology accuracy should treat the generated dimensions as a design specification and should measure the fabricated object after printing.

The current public detector is a single-camera estimator. The repository contains multi-camera benchmark scaffolding, but this should be treated as development material until the associated calibration and fusion API is surfaced in the package. Natural next steps include physical benchmark datasets, calibrated real-camera error reports, a documented multi-camera interface, and additional generated targets optimized for small objects, wide-baseline cameras, or robot end-effector mounting.

The most natural uses are robotic object tracking, calibration props, physical debugging targets, simulation-to-real visualization, and small lab workflows that need a printable six degree-of-freedom reference object with known geometry. AprilCube is particularly useful when the target may be viewed from

multiple sides or when a planar board would be awkward to attach to the object being tracked.

XI. AVAILABILITY

The source code, generated models, examples, tests, and this paper source are available at <https://github.com/younghyopark/aprilcube>. The repository is released under the MIT license.

ACKNOWLEDGMENT

This paper documents the AprilCube repository and the generated artifacts included with it.

REFERENCES

- [1] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marin-Jimenez, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [2] E. Olson, “AprilTag: A robust and flexible visual fiducial system,” in *Proc. IEEE International Conference on Robotics and Automation*, 2011.
- [3] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’ Journal of Software Tools*, 2000.
- [4] G. Terzakis and M. Lourakis, “A consistently fast and globally optimal solution to the perspective-n-point problem,” in *Proc. European Conference on Computer Vision*, 2020.
- [5] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.